

Inferring interventions from a fitted schedule

From a time-varying transmission rate back to a program, and on to another model

Fitting a model to case data often yields a piecewise-constant transmission rate (τ). `infer` turns the pair (baseline schedule, observed schedule) back into a program: one atom per maximal run where the observed value differs from baseline by a constant ratio (multiplicative algebras) or difference (additive algebras). The program can then be transported to a model with different target names and simulated.

```
using CategoricalInterventions
using AlgebraicPetri
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)
```

A fitted schedule

We construct one synthetically: a known program is applied to a constant baseline, and the result is rounded as a fit would be.

```
space = TargetSpace()
declare!(space, :beta, Affine(Multiplicative())); value_type=Float64)

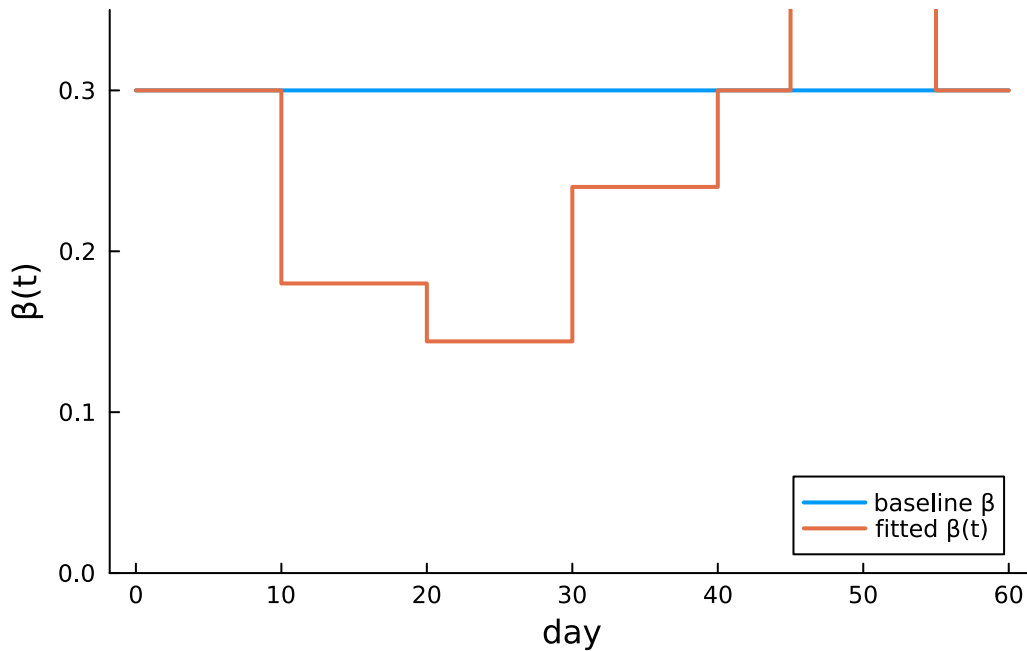
truth = @interventions space begin
    closure = scale(:beta, 0.6; during=10.0..30.0)
    masks    = scale(:beta, 0.8; during=20.0..40.0)
    reopen   = set(:beta, 0.36; during=45.0..55.0)
end
```

```

days = collect(0.0:1.0:60.0)
baseline = Dict{:beta => fill(0.30, length(days)))
fitted = Dict{:beta => round.(apply(truth, baseline, days)[:beta]; digits=6))

plot(days, baseline[:beta]; label="baseline ", xlabel="day", ylabel="  $\beta(t)$ ", ylims=(0, 0.35))
plot!(days, fitted[:beta]; label="fitted  $\beta(t)$ ", seriestype=:steppost)

```



Infer a program

```

inferred = infer(baseline, fitted, days; space=space)
inferred.atoms

```

4-element Vector{Atom}:

```

beta_1: Parameter:beta scale(0.6) on [10.0, 20.0)
beta_2: Parameter:beta scale(0.48) on [20.0, 30.0)
beta_3: Parameter:beta scale(0.8) on [30.0, 40.0)
beta_4: Parameter:beta scale(1.2) on [45.0, 55.0)

```

The overlap of the closure and the masks is recovered as one run with ratio 0.48, and the reopening, an absolute assignment in the original, is recovered as the ratio 1.2. This is the stated simplification: under a multiplicative algebra `infer` sees only ratios, so it returns the unique minimal program of scalings, not the original atoms.

```
apply(inferred, baseline, days)[:beta] == fitted[:beta]
```

```
true
```

Transport to another model

The inferred program names `:beta`; an AlgebraicPetri SIR names its infection rate `:inf`. `pushforward` renames along the target map, and the declaration table travels with it.

```
sir = LabelledPetriNet([:S, :I, :R], :inf => ((:S, :I) => (:I, :I)), :rec => (:I => :R))
model = Model(sir)
program = pushforward(inferred, Dict(:beta => :inf); space=model.space)
program.atoms
```

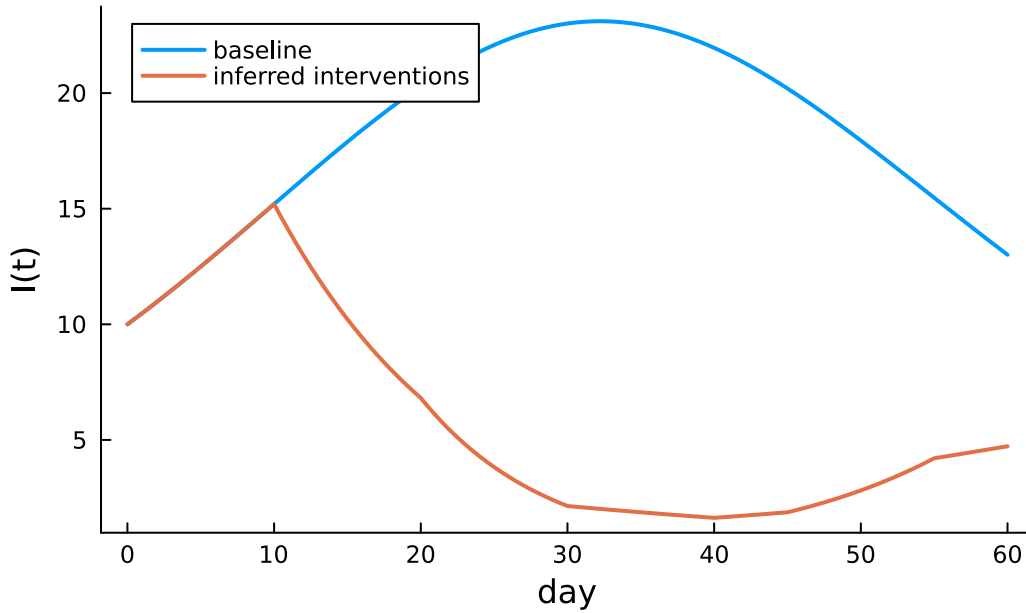
```
4-element Vector{Atom}:
```

```
beta_1: Parameter:inf scale(0.6) on [10.0, 20.0)
beta_2: Parameter:inf scale(0.48) on [20.0, 30.0)
beta_3: Parameter:inf scale(0.8) on [30.0, 40.0)
beta_4: Parameter:inf scale(1.2) on [45.0, 55.0)
```

```
u0 = LVector(S=990.0, I=10.0, R=0.0)
p0 = LVector(inf=0.30 / sum(u0), rec=0.25)
tspan = (0.0, 60.0)
with = simulate(model, program; u0, p0, tspan, alg=Tsit5(), saveat=0.5)
without = simulate(model, Program(model); u0, p0, tspan, alg=Tsit5(), saveat=0.5)

plot(without.t, without[2, :]; label="baseline", xlabel="day", ylabel="I(t)",
      title="Inferred program transported to the Petri net")
plot!(with.t, with[2, :]; label="inferred interventions")
```

Inferred program transported to the Petri net



Because the Petri net's inf is $/ \mathbb{N}$ rather than $/ \mathbb{R}$, the ratios apply unchanged: a multiplicative program is invariant under rescaling the baseline.

What the category theory bought

Applying the inferred program to the baseline returns the observed schedule on the grid (`infer_putget`): `infer` is the `get` of a lens whose `put` is `apply`. Renaming targets is functorial and, being injective here, preserves conflict-freeness (`pushforward_comp`, `conflictFree_pushforward_of_injOn`), so the program that explains the fit is, without further checking, a valid program on the new model. The Lean development proves the per-cell form of `infer_putget`; the run-merging done here is an implementation refinement that the property tests cover.